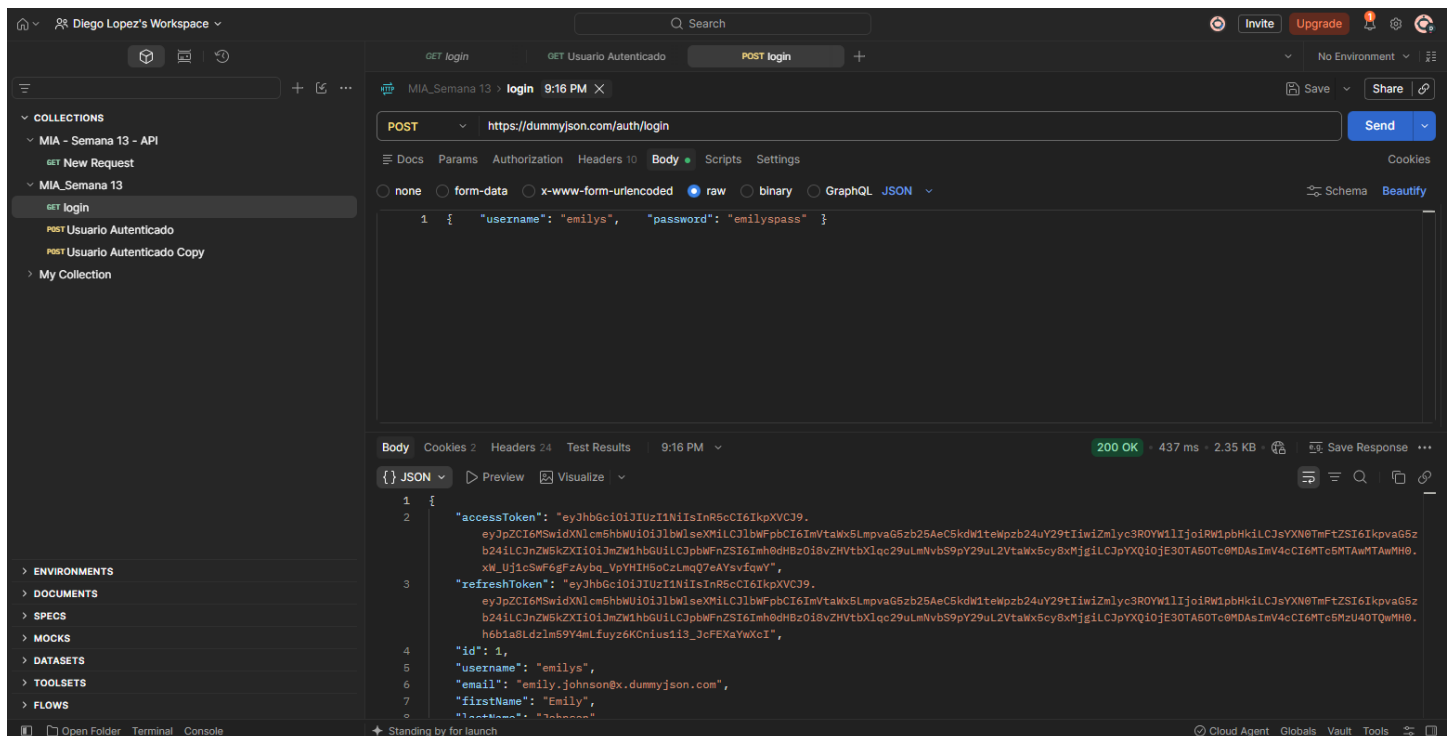
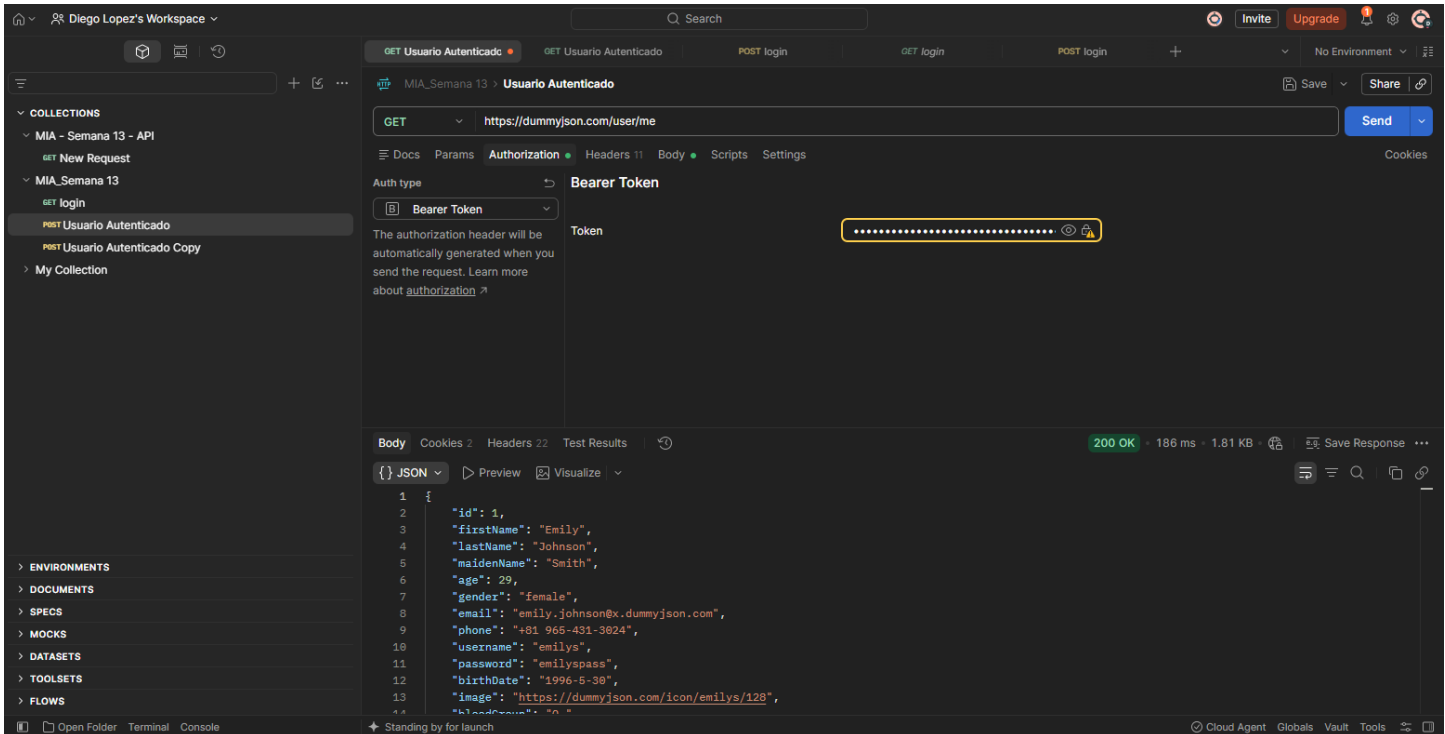






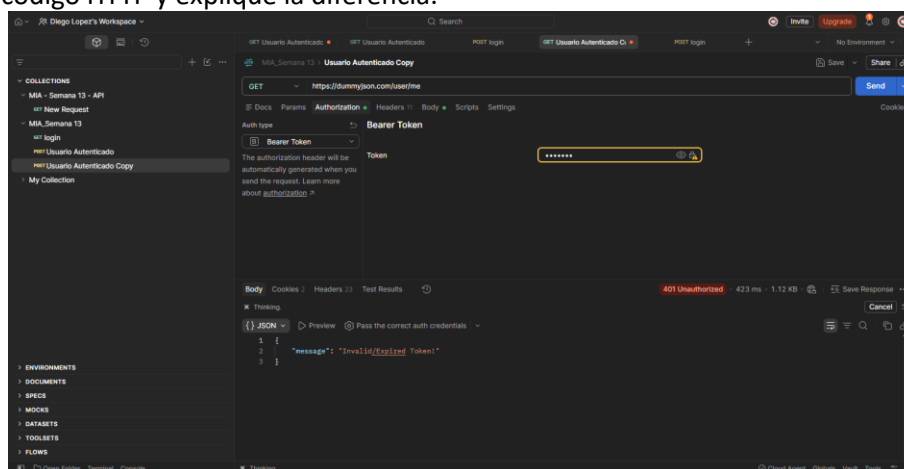
2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.



3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.





Con token válido (200 OK): La API devuelve la información privada y personalizada del usuario autenticado (por ejemplo: correo, roles, perfil, configuraciones).

Sin token (200 OK): La API responde exitosamente, pero el cuerpo del JSON cambia: devuelve información de carácter **público**, datos en modo "invitado/anónimo" o omite los campos sensibles que requieren sesión.

Con token erróneo (401 Unauthorized): La API rechaza la petición al detectar un intento fallido de validación de identidad.

La diferencia en el comportamiento del servidor se debe a cómo el backend maneja el *middleware* de seguridad:

1. **Sin Token (200 OK): Autenticación Opcional** El endpoint no exige autenticación obligatoria para responder. Al no detectar la cabecera Authorization, el servidor asume que el cliente es un usuario anónimo o público y procesa la solicitud devolviendo una vista básica sin error.
2. **Con Token Inválido (401 Unauthorized): Fallo de Credenciales** Cuando se envía una cabecera Authorization (aunque sea un token falso o expirado), el servidor intenta validar la firma y estructura del token JWT/Bearer. Al fallar la verificación, el filtro de seguridad intercepta la petición y responde inmediatamente con 401 Unauthorized, indicando que las credenciales presentadas no son válidas.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- a) Abra el proyecto C# de Azure Storage.
- b) En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- c) Busque la variable que contiene la Connection String.
- d) Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- a) En la carpeta principal del proyecto, cree un archivo llamado .env.
- b) Dentro del archivo agregue la variable:
`AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING`
- c) Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- d) Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- e) Compruebe que el programa continúa funcionando correctamente.
- f) **No publique el archivo .env en GitHub**, ya que contiene la credencial real.



8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

Repository secret added.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

[New repository secret](#)

Name	Last updated
AZURE_STORAGE_CONNECTION_STRING	now

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.